

Faces -- Answered Sections

These sections are fully derivable from the code already in the repo (q1a_perceptron_faces.py, q1b_neural_net_scratch_faces.py, q1c_neural_net_pytorch_faces.py, util_faces.py, q2q3_run_all_stats.py). Drop into the final report as-is or lightly edit for prose.

§2 Algorithm descriptions (faces)

2.1 Part 1(a): Binary Perceptron (from scratch)

Model and architecture. A single binary perceptron with one weight per input pixel. Weights are stored as a $(70, 60)$ matrix w and one scalar bias b . For an input image x of shape $(70, 60)$:

```
score s = sum(w * x) + b
predict 1 (face)    if s >= 0
predict 0 (not face) otherwise
```

Weights are initialised to zero and the bias to zero, so the first update is purely data-driven.

Loss / update rule. No explicit loss; the perceptron uses the mistake-driven update rule. For each training example (x, y) :

```
y_hat = predict(x)
if y_hat != y:
    w <- w + (y - y_hat) * x
    b <- b + (y - y_hat)
```

This is equivalent to: if the true label is 1 and we predicted 0, push w toward x ; if the true label is 0 and we predicted 1, push w away from x . The classifier sweeps the training set $\text{max_iterations} = 100$ times.

Features. Pixel-level binary features. Each character in the ASCII face image is mapped to 1 if it is + or #, else 0 (util_faces._parse_image_file). The perceptron consumes the raw $(70, 60)$ 2D grid via element-wise multiplication.

2.2 Part 1(b): 3-layer Neural Network (numpy from scratch)

Model and architecture. Fully connected feed-forward network:

```
input (4200) -> hidden1 (128, ReLU) -> hidden2 (64, ReLU) -> output (2, softmax)
```

Weights are initialised with He scaling

($W \sim N(0, \sqrt{2 / \text{fan_in}})$); biases at zero. The 70x60 image is flattened to a 4200-dim vector before entering the network.

Loss. Cross-entropy on a softmax over two output classes (face / not face). For a batch of N examples:

$$L = -(1/N) \sum_i \sum_k y_{ik} \log(\hat{y}_{ik})$$

where y_{ik} is the one-hot true label and $\hat{y}_{ik}$ is the softmax output.

Update rule. Vanilla mini-batch gradient descent (no momentum, no weight decay) at learning rate $\text{lr} = 0.01$:

$$W \leftarrow W - \text{lr} \, dL/dW, \quad b \leftarrow b - \text{lr} \, dL/db$$

Backpropagation derivation sketch. With ReLU on the two hidden layers and softmax+cross-entropy at the output, the gradients are:

Forward

$$\begin{aligned} z_1 &= X W_1 + b_1, & a_1 &= \text{ReLU}(z_1) \\ z_2 &= a_1 W_2 + b_2, & a_2 &= \text{ReLU}(z_2) \\ z_3 &= a_2 W_3 + b_3, & \hat{y} &= \text{softmax}(z_3) \end{aligned}$$

Output layer (softmax + cross-entropy collapse cleanly)

$$\begin{aligned} dz_3 &= (\hat{y} - y) / N \\ dW_3 &= a_2^T @ dz_3 \\ db_3 &= \text{sum}(dz_3, \text{axis}=0) \end{aligned}$$

Hidden layer 2 (ReLU')

$$\begin{aligned} da_2 &= dz_3 @ W_3^T \\ dz_2 &= da_2 * 1[z_2 > 0] \\ dW_2 &= a_1^T @ dz_2 \\ db_2 &= \text{sum}(dz_2, \text{axis}=0) \end{aligned}$$

Hidden layer 1 (ReLU')

$$\begin{aligned} da_1 &= dz_2 @ W_2^T \\ dz_1 &= da_1 * 1[z_1 > 0] \\ dW_1 &= X^T @ dz_1 \\ db_1 &= \text{sum}(dz_1, \text{axis}=0) \end{aligned}$$

The clean $dz_3 = (\hat{y} - y) / N$ form is the standard simplification you get when you compose softmax with cross-entropy. ReLU's derivative is the indicator $1[z > 0]$, applied element-wise. Implemented in `q1b_neural_net_scratch_faces.py:81-103`.

Features. Same pixel-level binary features as 1(a), but flattened to a 4200-dim vector via `util_faces.flatten_images`.

2.3 Part 1(c): 3-layer Neural Network (PyTorch)

Model and architecture. Identical topology to 1(b):

`nn.Linear(4200, 128) -> ReLU -> nn.Linear(128, 64) -> ReLU -> nn.Linear(64, 2)`

Default PyTorch `nn.Linear` initialisation (Kaiming uniform). The forward method returns raw logits; softmax is implicit inside the loss.

Loss. `nn.CrossEntropyLoss`, which composes log-softmax and NLL internally over the two-class output.

Update rule. Adam (`torch.optim.Adam`, `lr = 1e-3`) with default betas. Mini-batches of size 32, 100 epochs.

Features. Same pixel-level binary features, flattened to 4200 dims, packaged as `torch.float32` tensors (`flatten_images` then `torch.tensor`).

§3 Experimental protocol (faces)

The driver `q2q3_run_all_stats.py` runs the same protocol for every classifier and every training fraction:

1. **Training fractions.** `p` in {10, 20, 30, 40, 50, 60, 70, 80, 90, 100} (every 10%).
2. **Sampling.** For each `p`, draw `num_iterations` independent random subsamples of size `p%` of the training set (`np.random.choice(..., replace=False)`).
3. **Fresh classifier per draw.** A new instance of the classifier is constructed and trained from scratch on each subsample (no warm-starting).
4. **Evaluation.** Each trained classifier is evaluated on the **full test set** (`facetest / facetestlabels`).
5. **Reported statistics.** Per (`classifier`, `p`) cell we report `mean_train_time`, `mean_error`, and `std_error` across the `num_iterations` draws.
6. **Default `num\_iterations`.** 5 (overridable with `-i N` on `q2q3_run_all_stats.py`).

Hyperparameters used (faces).

Classifier	Hidden sizes	Activation	LR	Optimiser	Epochs / pass	Batch	Loss
Perceptron 1(a)	(n/a)	step	(n/a)	mistake-driven	100 passes	1 (online)	(no explicit loss)
Scratch NN 1(b)	128, 64	ReLU	0.01	SGD	100 epochs	32	softmax + CE
PyTorch NN 1(c)	128, 64	ReLU	1e-3	Adam	100 epochs	32	CrossEntropyLoss

Random seeding. No fixed seed is set in any of the face mains; each iteration genuinely draws a new subsample, which is what the `std_error` bars are meant to capture.

§4 Results -- protocol notes (numbers go in the template)

The two figures the report needs for faces:

1. **Mean test error vs training fraction**, with error bars of ± 1 std across iterations, one curve per classifier.
2. **Mean training time vs training fraction**, one curve per classifier (linear y-axis is fine; log-y if Adam dominates).

Source for both: `results.json` produced by

```
python3 q2q3_run_all_stats.py -o results.json
```

(or limit to face classifiers with

```
-w perceptron_faces scratch_faces pytorch_faces).
```

§5 Discussion -- claims that hold without running anything

A few framing points that hold regardless of the numbers:

- **Hypothesis space gap.** The perceptron is a single linear classifier on raw pixels; the two MLPs add two non-linear hidden layers. On a binary task as visually structured as faces, the perceptron is already a surprisingly strong baseline because the classes are roughly linearly separable in pixel space, but the MLPs should match or beat it once they have enough data.
 - **Optimiser gap between 1(b) and 1(c).** The two MLPs share an architecture but differ in optimiser (SGD with $\text{lr}=0.01$ vs Adam with $\text{lr}=1e-3$) and initialisation (He vs Kaiming uniform). Any systematic gap between 1(b) and 1(c) at the same training fraction is attributable to those two factors, not to capacity.
 - **Variance source.** The std bars conflate two independent sources of randomness: the training subsample draw and (for the NNs) the weight init / mini-batch shuffle. At small p the subsample term dominates; at $p = 100$ only the init / shuffle term remains.
 - **Time scaling.** Per-epoch cost for both NNs is $O(N)$ in dataset size, so training time should grow roughly linearly with p . Perceptron training time is also $O(N)$ per pass but with a much smaller constant. Expect the curves to be approximately linear with different slopes.
-

§6 External resources

- Standard textbook references for the softmax + cross-entropy gradient collapse and ReLU's subgradient (e.g. Goodfellow, Bengio, Courville, *Deep Learning*, ch. 6).

- PyTorch documentation for `nn.CrossEntropyLoss`, `nn.Linear`, and `torch.optim.Adam`.

- **AI tools used:** TODO -- list any LLM assistance here per the README integrity clause ("cite any external resources... that you drew on").